

Participant Welcome & How to Use This Document

Welcome to the 4-Hour Software Hackathon. This document is your official guide to the problem statements. You should select a track that aligns with your team's interest and expertise. Once chosen, plan to spend approximately 2.5–3 hours on building and the final hour on testing and polish. Our core philosophy is **Working Prototype over Slide Decks**: judges will prioritize functional code and live logic over aesthetic presentations.

Career Catalyst Club × GeeksforGeeks: 4-Hour Software Hackathon

Document 1: Official Problem Statements & Technical Reference Notes

Target Audience: Undergraduate Engineering Students | Duration: 4 Hours | Location: India

Executive Overview of Tracks

This document contains the official set of hackathon problem statements designed for a strict 4-hour rapid prototyping event, followed by the technical reference and operational notes for organizers and judges.

Primary Problem Statements (Official Selection)

[CC-GFG-01] AgentGuard: Autonomous AI Agent Runtime Security & Action Gateway

Difficulty: Intermediate **Theme:** Agentic AI & AI Security

The Real-World Context (In Simple Words): In 2026, autonomous AI agents are everywhere, but they can be dangerous. Enterprise workflows need an automated firewall to prevent rogue behavior.

The Core Problem: AI agents with tool-calling capabilities can accidentally delete databases or leak credentials due to prompt injection or reasoning loops.

Why This Problem Matters: In 2026, agentic AI deployment has accelerated, but runtime security guardrails remain immature. Enterprise and developer workflows require an automated firewall layer to prevent rogue agent behavior without degrading workflow speed.

Your Exact 4-Hour Mission (What to Build):

- 1. Build middleware to intercept tool-call JSON payloads.

- 2. Create a policy engine to classify actions as Green (approve), Amber (log), or Red (destructive).
- 3. Develop a web UI for real-time monitoring and human-in-the-loop authorization for Red actions.

Recommended Tech Stack & Free Resources: Python (FastAPI/Flask), Streamlit or React, Pydantic, JSON Schema validation.

4-Hour MVP Deliverable: Interceptor middleware that parses simulated agent tool-call JSON objects, blocks high-risk operations, and renders a live approval dashboard.

Expected 60–90 Second Live Demo: Show a benign read action passing instantly, followed by a destructive "drop_database_table" call that triggers a UI alert and waits for manual approval.

Organizer & Judging Notes:

- **Team Size:** 2–4 members.
- **Estimated Feasibility:** High (~2.5 hours build, 1.5 hours testing/polish).
- **Judging Focus:** Policy engine reliability, latency of JSON parsing, clear UI risk explanation.
- **What to Reject as Incomplete:** Static frontend mockups that do not intercept or parse live JSON payloads.
- **Bonus Features:** Regex-based prompt injection detection in arguments; webhook notification (Slack/Discord).

[CC-GFG-02] UPI-Shield: Contextual Digital Payment Scam & Coercion Detector

Difficulty: Beginner Theme: Fintech Security & NLP

The Real-World Context (In Simple Words): Scammers use psychological tricks like "electricity disconnection" to steal money via UPI, making traditional 2FA ineffective.

The Core Problem: Users willingly authorize fraudulent transactions under duress or deception.

Why This Problem Matters: Millions of digital payment users in India are vulnerable to social-urgency deception where traditional 2FA and MPIN checks fail because the user willingly authorizes the transaction under duress or deception.

Your Exact 4-Hour Mission (What to Build):

- 1. Interface to ingest SMS, WhatsApp, or payment notes.
- 2. NLP engine to detect urgency markers and unauthorized authority claims.
- 3. Output a Threat Meter and bilingual (English/Hindi) safety warnings.

Recommended Tech Stack & Free Resources: Python (FastAPI/Streamlit), Hugging Face zero-shot classifiers, Gemini free tier, Google Translate.

4-Hour MVP Deliverable: End-to-end web application that processes message text, flags deceptive triggers, and outputs bilingual safety cards.

Expected 60–90 Second Live Demo: Paste a scam message about a "verification refund"; the tool must flag it as High Risk and show the Hindi translation.

Organizer & Judging Notes:

- **Team Size:** 2–3 members (accessible to solo developers).
- **Estimated Feasibility:** Very High (~2 hours build, 2 hours polish).
- **Judging Focus:** Accuracy of scam trigger identification, explanation clarity for non-technical users.
- **What to Reject as Incomplete:** Trivial string equality matching (e.g., if "pin" in text) without generalizable heuristics or semantic analysis.
- **Bonus Features:** Parsing raw UPI intent strings (upi://pay?pa=...); OCR extraction from uploaded mobile screenshots.

[CC-GFG-03] CodeSanitizer: AI-Generated Code & Package Hallucination Auditor

Difficulty: Intermediate Theme: AI Engineering & DevSecOps

The Real-World Context (In Simple Words): Attackers register fake package names that AI often hallucinates, leading to supply chain attacks when developers copy AI code.

The Core Problem: AI assistants introduce insecure patterns like hardcoded keys or non-existent, malicious libraries.

Why This Problem Matters: Attackers actively register commonly hallucinated package names with malicious payloads on PyPI/npm. An automated pre-commit tool prevents supply chain attacks and security flaws before code enters production.

Your Exact 4-Hour Mission (What to Build):

- 1. Use Python AST to extract imports and dangerous function calls.
- 2. Verify libraries against the live PyPI API.
- 3. Generate a vulnerability report with secure code replacements.

Recommended Tech Stack & Free Resources: Python (ast, httpx), Streamlit or CLI.

4-Hour MVP Deliverable: Interactive code auditor that validates imports against PyPI in real time and detects critical AST vulnerability nodes.

Expected 60–90 Second Live Demo: Upload code with a fake library and an unescaped SQL query; show the tool flagging the import and suggesting a parameterized fix.

Organizer & Judging Notes:

- **Team Size:** 2–3 members.
- **Estimated Feasibility:** High (~2.5 hours build). Built-in Python ast eliminates complex dependency overhead.
- **Judging Focus:** AST traversal depth, asynchronous non-blocking PyPI lookups, practical remediation hints.
- **What to Reject as Incomplete:** Basic regex string searches that do not parse the code structure via AST.
- **Bonus Features:** Auto-generation of .env.example when hardcoded secrets are found; one-click sanitized patch download.

[CC-GFG-04] CivicEye: Emergency Corridor Obstruction Sentinel

Difficulty: Intermediate Theme: Smart Cities & Computer Vision

The Real-World Context (In Simple Words): Ambulances get stuck because people park in emergency zones. Cities need a cheap way to detect this using

existing cameras.

The Core Problem: Obstructions in rapid transit lanes delay life-saving emergency responses.

Why This Problem Matters: Rapid emergency response saves lives. Municipalities need a cost-effective system that operates on standard CCTV hardware without expensive proprietary sensor grids.

Your Exact 4-Hour Mission (What to Build):

- 1. Process video clips using an object detection pipeline.
- 2. Deploy YOLOv8 nano to track vehicles on CPU.
- 3. Define virtual boundaries and trigger an alarm if a vehicle stops for >5 seconds.
- Obstruction timer that logs an incident and triggers visual/audible alarms when a vehicle stays inside the zone for over 5 seconds.

Recommended Tech Stack & Free Resources: Python, OpenCV, Ultralytics YOLOv8 nano, Streamlit.

4-Hour MVP Deliverable: Computer vision prototype running on CPU with interactive zone definition and time-based collision alerts.

Expected 60–90 Second Live Demo: Play traffic video where a car stops in a red zone; show the timer counting up and the alert firing with a captured image.

Organizer & Judging Notes:

- **Team Size:** 3–4 members.
- **Estimated Feasibility:** Medium-High (~3 hours build). Using YOLOv8 nano ensures 15–25 FPS on CPU.
- **Judging Focus:** Stable tracking FPS on CPU, accurate polygon collision math, zero false triggers on moving traffic.
- **What to Reject as Incomplete:** Plain object detection on video without virtual zone boundary checks or time-tracking logic.
- **Bonus Features:** Snapshot auto-save of offending vehicle; dynamic UI sliders for zone coordinates.

[CC-GFG-05] RAG-Truth: Real-Time Hallucination & Grounding Auditor

Difficulty: Intermediate Theme: Trustworthy AI & NLP

The Real-World Context (In Simple Words): AI often makes up "facts" that aren't in its source documents. This creates massive liability in legal or campus policy tasks.

The Core Problem: Ungrounded AI answers are difficult for users to verify manually without a claim-by-claim breakdown.

Why This Problem Matters: In legal, academic, and enterprise deployments, ungrounded

AI answers cause severe liability. Developers need lightweight, objective verification layers that score factual adherence.

Your Exact 4-Hour Mission (What to Build):

- 1. Interface to accept Query, Source Doc, and AI Answer.
- 2. Split response into individual claims.
- 3. Verify claims using cross-encoders and highlight results (Green/Red).
- UI rendering an aggregate Faithfulness Score (0–100%) and color-coded text highlighting (Green = Supported, Red = Hallucinated).

Recommended Tech Stack & Free Resources: Python, Sentence-Transformers, Hugging Face, Streamlit.

4-Hour MVP Deliverable: Web app that accepts a reference text and generated answer, decomposes claims, and visually highlights hallucinations sentence by sentence.

Expected 60–90 Second Live Demo: Input a policy and a summary with one lie; show the tool highlighting the lie in red and giving a Faithfulness Score.

Organizer & Judging Notes:

- **Team Size:** 2–4 members.
- **Estimated Feasibility:** High (~2.5 hours build). Pairwise sentence similarity requires minimal boilerplate.
- **Judging Focus:** Claim segmentation accuracy, clear visual UI attribution, low

verification latency.

- **What to Reject as Incomplete:** Passing the whole text to an LLM with a single prompt ("Is this true?") without sentence-level decomposition and highlighting.
- **Bonus Features:** Source sentence mapping indicating which reference sentence verified the claim; sensitivity threshold slider.

[CC-GFG-06] VaaniAccess: Dialect-Adaptive Public Scheme Voice Navigator

Difficulty: Beginner Theme: Voice AI & Accessibility

The Real-World Context (In Simple Words): Many citizens can't access welfare because the forms are too complex. They need a simple way to ask for help using their voice.

The Core Problem: Literacy and bureaucratic language barriers prevent people from finding schemes like PM-Kisan.

Why This Problem Matters: True digital inclusion in India requires spoken-language interfaces that convert complex government entitlements into accessible, conversational guidance.

Your Exact 4-Hour Mission (What to Build):

- 1. Use Web Speech API for Hindi voice input.
- 2. Map colloquial speech to eligibility criteria (e.g., "farmer").
- 3. Match schemes and speak back instructions in Hindi.
- Speech synthesis reading back simple, 3-step actionable instructions in Hindi.

Recommended Tech Stack & Free Resources: JS Web Speech API, Python (FastAPI), gTTS.

4-Hour MVP Deliverable: Voice-in, voice-out web prototype that maps spoken Hindi queries to eligible welfare schemes with spoken instructions.

Expected 60–90 Second Live Demo: Speak "Main college student hoon" into the mic; show the app matching a scholarship and reading out steps in Hindi.

Organizer & Judging Notes:

- **Team Size:** 2–3 members.
- **Estimated Feasibility:** Very High (~2 hours build). Web Speech API requires no external API keys.
- **Judging Focus:** Speech recognition accuracy on Indian accents, scheme matching precision, clarity of spoken output.
- **What to Reject as Incomplete:** Text-only chatbots without functional speech recognition or spoken audio playback.
- **Bonus Features:** Downloadable visual document checklist; fallback text input toggle for noisy environments.

Additional Extended Problem Statements (Reference Archive)

[CC-GFG-07] DeepFakeSentry: Lightweight Multimodal Synthetic Media Detector

Difficulty: Advanced Theme: Multimodal AI & Digital Forensics

The Real-World Context (In Simple Words): Fake videos and voice clones are used for extortion. We need a way to spot digital artifacts that humans might miss.

The Core Problem: AI-generated media contains subtle anomalies like frequency cutoffs or flickering boundaries.

Your Exact 4-Hour Mission (What to Build): Build a pipeline to inspect clips for spectrogram anomalies and generate an Authenticity Score.

Expected 60–90 Second Live Demo: Show an audio spectrogram highlighting the robotic silence patterns and frequency drop-offs.

Recommended Tech Stack & Free Resources: Python, Librosa, OpenCV,

Streamlit.

Suggested Stack: Python, Librosa, OpenCV, Streamlit. **MVP:** Audio spectrogram generator highlighting abrupt frequency drop-offs and robotic silence patterns.

[CC-GFG-08] EcoWatt: Real-Time Campus Micro-Grid Energy Optimizer

Difficulty: Beginner **Theme:** Climate Tech & Resource Optimization

The Real-World Context (In Simple Words): Colleges pay too much for power because they use heavy machines when solar power isn't peaking.

The Core Problem: Unsynchronized loads (like ACs) lead to high tariffs and wasted solar energy.

Your Exact 4-Hour Mission (What to Build): Build a scheduler that moves deferrable loads to peak solar hours.

Expected 60–90 Second Live Demo: Show an interactive 24-hour schedule that calculates monetary and carbon savings.

Recommended Tech Stack & Free Resources: Python (scipy.optimize/pulp),

Streamlit, Plotly.

Suggested Stack: Python (scipy.optimize or pulp), Streamlit, Plotly. **MVP:** Interactive 24-hour schedule optimizer showing monetary and carbon savings.

[CC-GFG-09] GitPulse: Git Pre-Commit Secret Interceptor & Auto-Remediator

Difficulty: Beginner **Theme:** DevSecOps & Security

The Real-World Context (In Simple Words): Accidentally uploading API keys to GitHub can lead to hacked cloud accounts in minutes.

The Core Problem: Hardcoded tokens in commits are a massive security risk that needs automated removal.

Your Exact 4-Hour Mission (What to Build): Build a scanner to catch high-entropy tokens and replace them with environment variables.

Expected 60–90 Second Live Demo: Run a code diff scan that detects a leak and produces an instant sanitized patch.

Recommended Tech Stack & Free Resources: Python/Node.js, Shannon Entropy

calculation, Streamlit.

Suggested Stack: Python/Node.js, Shannon Entropy calculation, Streamlit. **MVP:** Code diff scanner that detects leaks and produces an instant sanitized patch.

[CC-GFG-10] PhishLens: Visual Brand Impersonation & Domain Mismatch Detector

Difficulty: Intermediate **Theme:** Computer Vision & Security

The Real-World Context (In Simple Words): Phishing sites use real logos to trick people. We need tools that look at the page like a human does to spot mismatches.

The Core Problem: Websites evade text-based filters by copying brand visuals onto unauthorized domains.

Your Exact 4-Hour Mission (What to Build): Build a tool to compare webpage screenshots (logos) against the actual URL.

Expected 60–90 Second Live Demo: Show an analyzer flagging a site using a brand logo on an unauthorized domain.

Recommended Tech Stack & Free Resources: Python, OpenCV, Multimodal

Vision API, Streamlit.

Suggested Stack: Python, OpenCV, Multimodal Vision API, Streamlit. **MVP:** Image + URL analyzer that highlights brand logos and flags unauthorized domains.

[CC-GFG-11] MediScript: Prescription Digitizer & Drug Conflict Checker

Difficulty: Intermediate **Theme:** Healthcare Technology & Multimodal AI

The Real-World Context (In Simple Words): Bad handwriting on prescriptions can lead to taking the wrong medicine, which is dangerous.

The Core Problem: Illegible scripts and uncoordinated drug regimens cause adverse reactions.

Your Exact 4-Hour Mission (What to Build): Build an OCR tool to parse prescription images and check for drug-drug interactions.

Expected 60–90 Second Live Demo: Digitize a prescription image and show the tool flagging a contraindication alert.

Recommended Tech Stack & Free Resources: Python, EasyOCR/Gemini Vision,

clinical dictionary, Streamlit.

Suggested Stack: Python, EasyOCR/Gemini Vision API, clinical lookup dictionary, Streamlit. **MVP:** Digitizer extracting drug names and flagging contraindication alerts.

[CC-GFG-12] SkillBridge: Employability Gap Analyzer & Custom Coding Assessment Engine

Difficulty: Beginner Theme: EdTech & Developer Productivity

The Real-World Context (In Simple Words): It's hard to know exactly what skills you're missing for a job. This tool gives you a personalized test to practice.

The Core Problem: Students lack a clear bridge between their current resume and specific job requirements.

Your Exact 4-Hour Mission (What to Build): Build a resume vs. JD comparator that generates an interactive coding challenge evaluated live.

Expected 60–90 Second Live Demo: Input a resume and JD; show the tool generating a custom GFG-style coding question and running it in-browser.

Recommended Tech Stack & Free Resources: Python, Streamlit, Pyodide, Free

LLM API.

Suggested Stack: Python, Streamlit, Pyodide (in-browser Python), Free LLM API. **MVP:** Resume vs. JD comparator that generates an interactive coding question evaluated live.